

初学者向け基礎知識資料

PatchCore・層化コアセット・GNN・高速化を理解するために

対象研究： オートリベッタ穿孔後の切粉・異物検知

基盤手法： PatchCore / SaikuPatchCore

文書版： v2.0

作成日： 2026 年 8 月 5 日

目次

1	この資料の目的	2
2	異常検知とは何か	2
3	画像・画素・パッチ	2
4	CNN と特徴量	2
5	PatchCore の仕組み	3
6	memory bank と coreset	3
6.1	なぜ層化が必要か	3
7	正常モードとは何か	4
8	PCA と IPCA	4
9	近似最近傍探索	5
10	GNN とグラフの基礎	5
11	なぜ全パッチへ GNN を使わないのか	5
12	評価指標の読み方	5
13	データリーク	6
14	高速化の考え方	6
15	実験ログの読み方	7
16	研究を進める順序	7

1 この資料の目的

本資料は、オートリベッタ穿孔後の切粉・異物検知研究を進めるために必要な基礎知識を、初学者向けに整理するものである。数式は最小限にし、何を入力し、どの処理を行い、何が出力されるかを中心に説明する。

2 異常検知とは何か

通常の画像分類は、正常・切粉・傷などのラベル付き画像からクラス境界を学習する。一方、工業異常検知では異常の種類を事前にすべて収集できないため、正常画像のみから「正常らしさ」を構築し、そこから離れた画像や領域を異常とみなすことが多い。

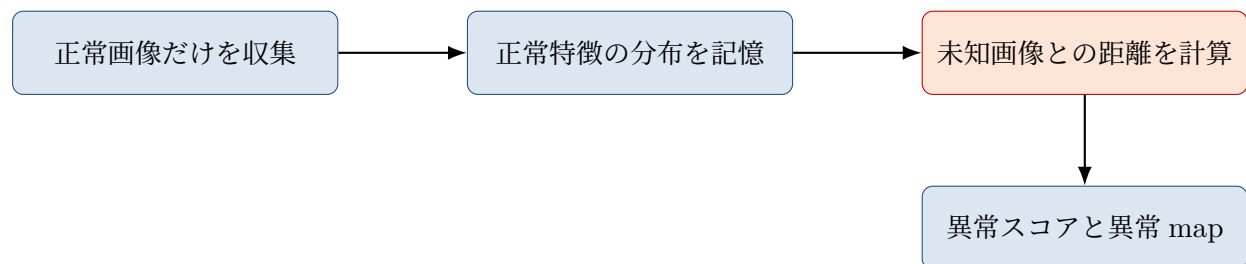


図 1: one-class 異常検知の基本概念

3 画像・画素・パッチ

画像は画素の格子であり、各画素は RGB 等の値を持つ。CNN へ画像を入力すると、元画像より小さい格子状の特徴 map が得られる。特徴 map 上の各位置を**パッチ特徴**として扱えば、画像全体ではなく局所領域ごとの異常度を計算できる。

局所異常で重要な点

切粉が画像全体に対して小さい場合、画像レベルの判定が正しくても、異常 map が切粉位置へ反応しているとは限らない。したがって、Image AUROC だけでなく、Pixel AP, AUPRO, 異常サイズ別 Recall, FP 領域を確認する必要がある。

4 CNN と特徴量

CNN は画像から輪郭、模様、形状などを段階的に抽出する。浅い層は細かいテクスチャを保持し、深い層は意味的情報を多く含む傾向がある。PatchCore では ImageNet 等で事前学習された CNN を固定し、KHI 画像を入力して得た中間特徴を利用する。これは KHI 画像で CNN を再学習することとは異なる。

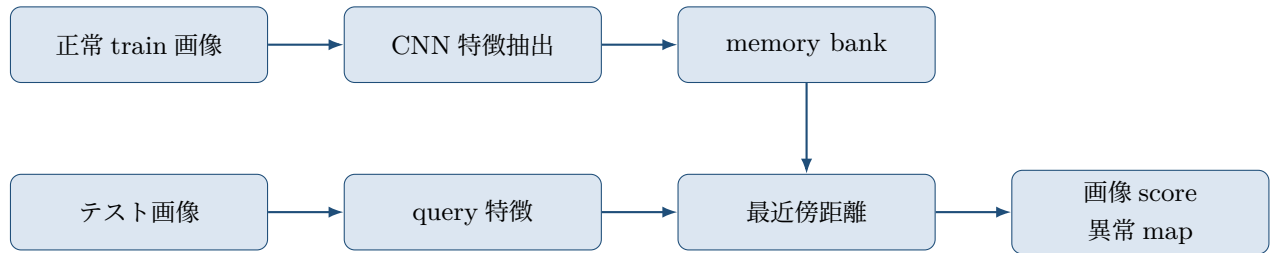
表 1: 特徴層の直感的な違い

層	得意な情報	注意点
浅い層	エッジ, 細かな質感, 微小切粉	反射やノイズにも反応しやすい
中間層	局所形状と質感の両方	PatchCore で利用しやすい
深い層	物体の意味, 広い文脈	微小異常が薄れる可能性

5 PatchCore の仕組み

PatchCore は, 正常画像のパッチ特徴を memory bank へ保存し, テストパッチと最も近い正常特徴との距離を異常度とする. テスト画像 x のパッチ特徴集合を $P(x)$, 正常 bank を M とすると, 代表的な画像スコアは次で表される.

$$s(x) = \max_{z_i \in P(x)} \min_{m_j \in M} \|z_i - m_j\|_2. \quad (1)$$



6 memory bank と coresets

正常パッチをすべて保存すると, 画像数と解像度の増加に伴い memory bank が巨大になる. 特徴数を N , 特徴次元を d , float32 を用いる場合, 容量は概ね

$$\text{Memory} \simeq 4Nd \quad [\text{bytes}] \quad (2)$$

となる. coresets は, 正常特徴全体を代表する少数点を選び, bank を圧縮する方法である.

6.1 なぜ層化が必要か

通常の global coresets では, 多数派の正常外観が多く残り, 稀な正常外観が消える可能性がある. 例えば, 明るいワークが全体の 3% しかない場合, bank 圧縮後にその特徴がほとんど残らず, 明るい正常画像を異常と判定する危険がある.

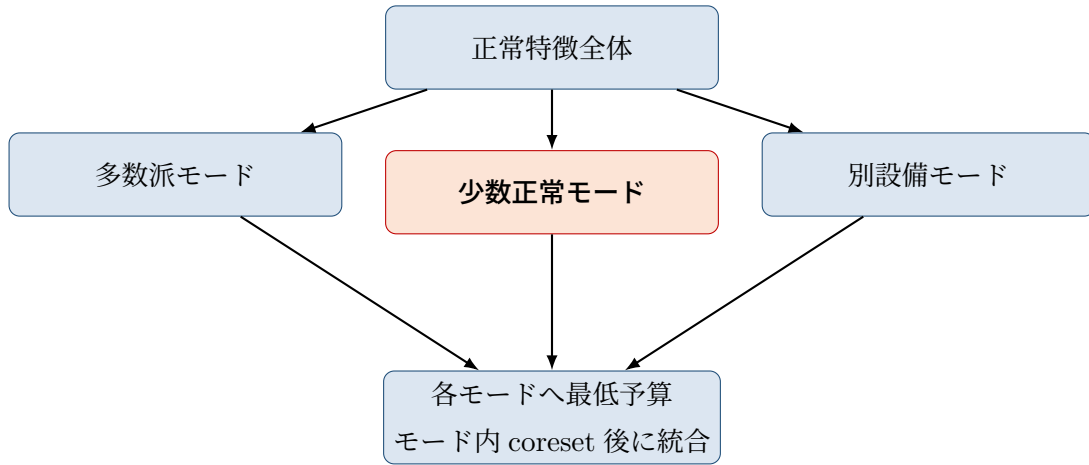


図 3: 正常モード保存型層化 coreset の考え方

7 正常モードとは何か

正常モードは、同じ正常ラベルの中にある異なる外観状態である。KHI 画像では、明度、反射、ワーク色、設備番号、撮影系列、孔の位置、半径方向の位置などが候補となる。

表 2: 正常モードの定義方法

方式	例	長所・注意点
明示属性	#6/#7, 明度分類, 撮影日	解釈しやすいが, 人手定義に依存
潜在 cluster	PCA 後 k-means 等	未知の外観差を発見できるが, cluster の意味確認が必要
hybrid	設備別に cluster を推定	既知差と潜在差を両立するが, mode 数が増えやすい

8 PCA と IPCA

PCA は、特徴のばらつきをよく表す方向を残して次元を減らす。特徴次元を d から k へ減らし、coreset 率を p とすると、bank 容量は概ね

$$\text{Memory}_{\text{compressed}} \simeq 4pNk \quad (3)$$

となる。IPCA は特徴を小分けに読みながら PCA を近似するため、大規模データで RAM を抑えやすい。ただし、異常を識別する微細な方向まで削除しないよう、Pixel AP や small-defect Recall を確認する。

9 近似最近傍探索

表 3: 最近傍探索方式の違い

方式	特徴	用途
FlatL2	全点との厳密距離	精度基準. bank が大きい と遅い
IVFFlat	粗い cluster を選んで探索	速度と精度の調整が容易
HNSW	graph をたどって探索	CPU で高速な候補
IVFPQ	圧縮 code で検索	省メモリだが v1 安定後に追加

10 GNN とグラフの基礎

グラフはノードとエッジからなる. 本研究では, パッチまたは正常 prototype をノードとし, 空間的に近い, 特徴が近い, 同じ正常モードである, 極座標上で隣接する, という関係をエッジで表す.

一般的な message passing は, ノード i の特徴 h_i を周辺ノード $\mathcal{N}(i)$ から更新する処理である.

$$h_i^{(l+1)} = \sigma \left(W_0 h_i^{(l)} + \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W_1 h_j^{(l)} \right). \quad (4)$$

周囲と整合している反射は正常, 孤立した切粉は異常, という関係を利用できる可能性がある. 一方, 層を深くしすぎると異常特徴が周囲へ平均化されるため, 本研究では 1-2 層を基本とする.

11 なぜ全パッチへ GNN を使わないのか

全パッチ graph はノード数とエッジ数が多く, 速度とメモリの負担が大きい. そこで, まず高速な PatchCore で全パッチを走査し, 異常度上位だけを graph で再評価する. これは空港の全員検査と追加検査の関係に近い.

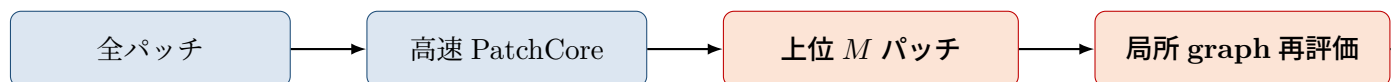


図 4: 二段階の疎 graph 推論

12 評価指標の読み方

指標	何を見るか	注意点
Image AUROC	正常・異常順位の分離	閾値運用や異常数の不均衡を直接示さない
Image AP	異常を上位へ集める能力	異常数が少ない条件で AUROC より有用
Pixel AUROC	正常画素・異常画素の順位	正常画素が非常に多く、高く出やすい
Pixel AP	異常画素の precision-recall	微小異常評価で重要
AUPRO	異常領域ごとの重なり	局在性能の標準指標候補
FP/1000 good	正常 1000 枚当たり誤警報	工場停止や人手確認負荷に直結
worst-mode FPR	最も弱い正常 mode の FPR	層化 coreset の主要指標
p95 latency	95% 画像が収まる時間	平均より工場タクト評価に向く

13 データリーク

データリークとは、本来見てはいけない test 情報を学習や設定選択に利用することである。同じ撮影系列の類似画像が train と test へ分かれた場合も、見かけ上高い性能になる危険がある。

- PCA fit, normal mode 推定, memory bank は train good だけで行う。
- 閾値と Optuna の目的値は validation で決める。
- test は最終条件を決めた後に一度だけ評価する。
- SHA-256 と group key で重複・系列混入を確認する。

14 高速化の考え方

Python を C++ へ変える前に、どこが遅いかを計測する。CNN, FAISS, OpenCV, NumPy の主要演算は内部で C/C++ や CUDA を利用するため、Python で呼んでも計算本体は高速である場合が多い。

高速化の優先順位は、計算対象を減らす、bank 数を減らす、特徴次元を減らす、ANN を使う、GNN 対象を減らす、推論 backend を最適化する、最後に必要部分だけ C++ 化する、の順が基本である。

表 5: 工場推論の候補構成

構成	長所	適用時期
Python + PyTorch	研究実装が速い	baseline・手法開発
Python + ONNX Runtime	CPU/GPU で移植しやすい	モデル確定後
Python + OpenVINO	Intel CPU 最適化候補	工場 CPU 評価
TensorRT	NVIDIA GPU で高速	GPU 導入時
C++ + runtime	制御統合・低 jitter	Python 版が要件未達の場合のみ

15 実験ログの読み方

実験が成功したかは、最後の AUROC だけで判断しない。次を順に確認する。

1. 画像枚数, 正常・異常枚数, GT 対応数は意図どおりか。
2. 使用した設定, seed, device, 解像度, bank 数は何か。
3. 各ステージは SUCCESS, CACHED, SKIPPED, FAILED のどれか。
4. Image/Pixel 指標と FP/FN かどうか。
5. 異常 map は実際の切粉へ反応しているか。
6. どの正常モードで誤検知したか。
7. 総時間と各工程時間, RAM, GPU memory かどうか。

16 研究を進める順序

1. PatchCore baseline を再現する。
2. KHI 正常画像の特徴を整理する。
3. 正常モードが本当に存在するか確認する。
4. global coreset で少数モードが消えるか測る。
5. 層化 coreset だけを比較する。
6. graph だけを比較する。
7. 両者を統合する。
8. 精度を保ったまま軽量化する。
9. #6, #7, 公開金属データへ展開する。

参考文献

- [1] K. Roth et al., “Towards Total Recall in Industrial Anomaly Detection,” CVPR, 2022.
- [2] X. Jiang et al., “SoftPatch: Unsupervised Anomaly Detection with Noisy Data,” NeurIPS, 2022.
- [3] W. Hamilton et al., “Inductive Representation Learning on Large Graphs,” NeurIPS, 2017.
- [4] K. Batzner et al., “EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies,” WACV, 2024.